

Materia: 37761-ARQUITECTURA DE SOFTWARE

Nombre: Lisbeth Amariles Yupangui Aguirre

Tema: Arquitectura Limpia

Profesor: Ing. ALVARO DANILO UYAGUARI UYAGUARI

Fecha: 23 de octubre de 2026

Resumen

Se propone evolucionar la aplicacion monolitica de facturacion hacia una arquitectura en la que las reglas estables del negocio permanezcan en el centro y los detalles de entrega y almacenamiento se conecten mediante adaptadores. Una API REST y un consumidor AMQP traducen sus respectivos formatos al mismo contrato de entrada; ambos invocan un unico caso de uso, CrearFacturaUseCase. El caso de uso coordina el proceso, mientras el dominio valida fraude y politica fiscal antes de solicitar el almacenamiento a traves de una abstraccion.

La sustitucion de PostgreSQL por Oracle o por un repositorio en memoria se resuelve en infraestructura mediante seleccion de adaptador y configuracion. La aplicacion depende de la interfaz FacturaRepository, no de SQL, JDBC ni de un proveedor. Asi, el flujo de negocio y sus pruebas pueden ejecutarse sin un motor real. Esta propuesta satisface el criterio central del enunciado: poder cambiar interfaz, broker o persistencia sin alterar las capas de dominio y aplicacion.

Alcance y honestidad academica

El PDF recibido plantea un trabajo y sus criterios, pero no incluye repositorio, codigo ni evidencias de ejecucion del equipo. Por ello, este informe describe un diseno de referencia y un plan verificable; no afirma que el codigo, las pruebas automatizadas, la conexion a bases reales o la implementacion AMQP ya se hayan ejecutado. Para entregar como memoria de un desarrollo realizado, deben incorporarse las evidencias reales del proyecto y sustituirse los campos de portada.

Decision arquitectonica principal: un puerto de entrada definido por la aplicacion y dos adaptadores simetricos (REST y AMQP) que lo llaman. El protocolo termina en el adaptador; no atraviesa el caso de uso.

1. Contexto, problema y objetivo

El sistema de partida es una aplicacion monolitica simple de consola. El cambio solicitado no consiste solo en agregar endpoints: busca mostrar que la logica de facturacion resiste cambios tecnologicos sin duplicar reglas ni ligar el negocio a una interfaz o base de datos. Se distinguen tres presiones de cambio: entrada sincrona o asincrona, politicas criticas de negocio y proveedores de persistencia diferentes.

Presion de cambio	Riesgo en un monolito acoplado	Respuesta propuesta
REST y mensajeria	Duplicar validacion y creacion en cada canal.	Adaptadores simetricos sobre CrearFacturaUseCase.
Fraude y fiscalidad	Reglas dispersas en UI, controladores o SQL.	Reglas de dominio ejecutadas por cada invocacion.
PostgreSQL u Oracle	Tipos, consultas o driver se filtran al negocio.	Puerto de repositorio y adaptadores seleccionables.

2. Arquitectura propuesta

La regla de dependencias apunta hacia las politicas internas: la infraestructura conoce y satisface contratos internos, pero el dominio y la aplicacion no importan frameworks, protocolos ni drivers. La inversion de dependencias permite que el flujo de control llame hacia afuera sin que la direccion del codigo se vuelva hacia afuera [1].

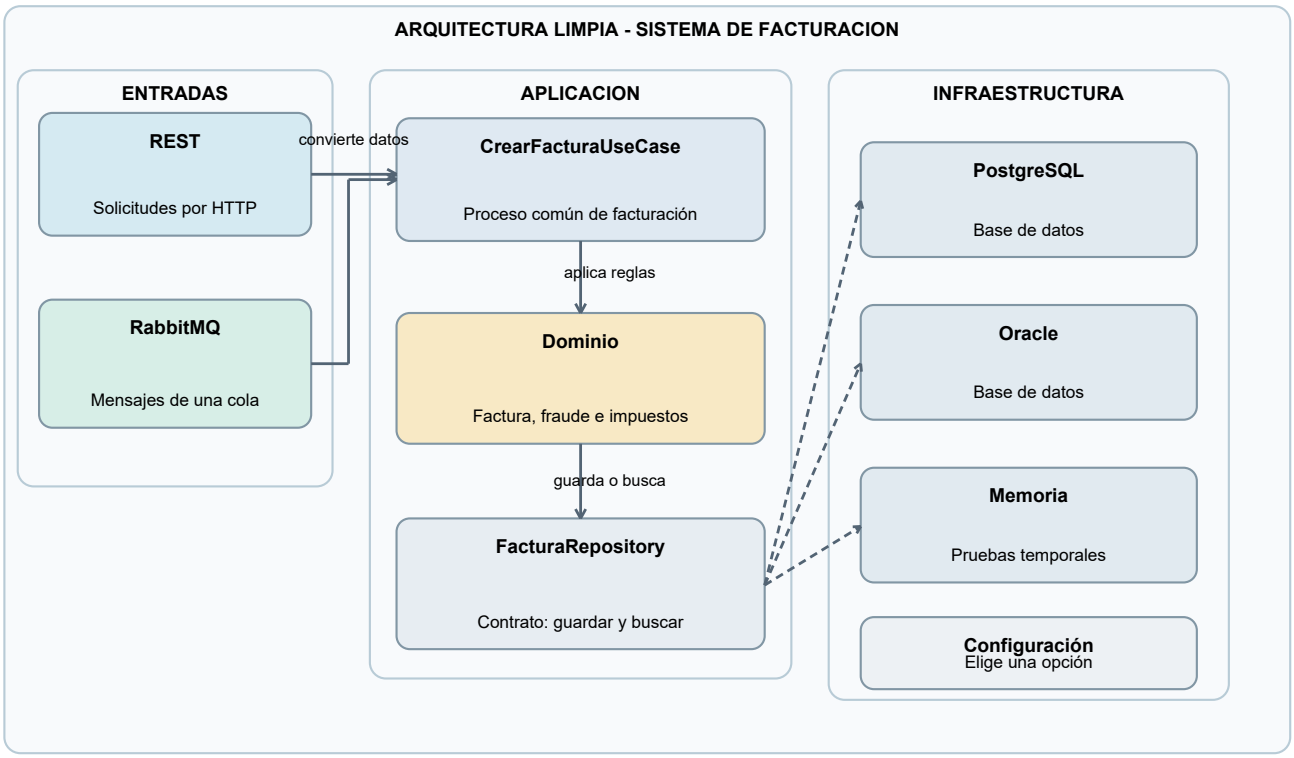


Figura 1. Diagrama de referencia. Las flechas muestran el flujo de invocacion; los adaptadores implementan o consumen contratos definidos hacia el interior.

Capas y responsabilidades

Capa	Elementos	Responsabilidad y limite
Dominio	Factura, LineaFactura, ClienteId, regla antifraude y politica fiscal.	Expresar invariantes y decisiones de negocio. Sin HTTP, AMQP, SQL, ORM, JDBC ni configuracion de proveedores.
Aplicacion	CrearFacturaUseCase, CrearFacturaCommand, resultado y puertos necesarios.	Coordinar el caso de uso, llamar las reglas del dominio y solicitar persistencia mediante contrato. Sin conocer el canal de entrada.
Adaptadores de interfaz	Controlador REST, consumidor AMQP, mapeadores DTO.	Convertir formatos externos a comandos internos y resultados a respuestas/acks propios del protocolo.
Infraestructura	Adaptadores de repositorio, driver, conexion, configuracion y ensamblaje.	Implementar persistencia y seleccionar proveedor. Gestionar recursos tecnicos, transacciones y observabilidad.

Nota de modelado

El enunciado llama contrato de persistencia al contrato que define el dominio. En este diseño, el contrato de almacenamiento requerido por el caso de uso se declara en la frontera interna (por ejemplo, dentro del paquete de aplicacion o dominio, segun la organizacion del proyecto), y es implementado por infraestructura. El punto que se debe defender es que la abstraccion pertenece al interior y los adaptadores concretos al exterior; ningun contrato menciona clases o tipos de un proveedor.

3. Punto de entrada compartido

El contrato de entrada pertenece a la capa de aplicacion. Los clientes del sistema no lo consumen directamente: un cliente HTTP habla REST y un productor habla AMQP. Cada adaptador transforma ese formato externo en un comando canonico y llama la misma interfaz de caso de uso. Esta es la identificacion clave que debe explicarse en la defensa.

Contrato ilustrativo (pseudocodigo independiente de lenguaje)

```
interface CrearFacturaUseCase {
CrearFacturaResult ejecutar(CrearFacturaCommand comando);
}

record CrearFacturaCommand(
String idOperacion,
ClienteId clienteId,
List<LineaFacturaCommand> lineas,
String moneda
);

interface FacturaRepository {
Optional<Factura> buscarPorIdOperacion(String idOperacion);
void guardar(Factura factura);
}
```

La idempotencia se incluye como atributo del comando porque REST puede reintentarse y el consumidor AMQP puede recibir nuevamente un mensaje. El contrato no contiene HttpRequest, HttpStatus, propiedades RabbitMQ, Connection, EntityManager ni tipos SQL. Los DTO de transporte pueden variar sin modificar el caso de uso.

Responsabilidades de entrada y salida

Elemento	Entrada	Salida
REST	Validar sintaxis, autenticacion/autorizacion de transporte, mapear JSON a comando.	Mapear resultado a codigo y cuerpo HTTP; errores a respuesta HTTP estable.
AMQP	Deserializar mensaje, validar envoltura y mapear payload a comando.	Confirmar ack tras procesamiento exitoso; rechazar/reencolar segun categoria de error y politica.

Flujo sincronico REST

1. El controlador recibe POST /facturas y transforma el cuerpo en CrearFacturaCommand. 2. Invoca CrearFacturaUseCase.ejecutar. 3. El caso de uso busca duplicados por idOperacion, construye el agregado y pide al dominio validar fraude y calcular politica fiscal. 4. Si las reglas aprueban, guarda por FacturaRepository. 5. El presentador REST convierte el resultado en respuesta HTTP, por ejemplo 201; conflictos de idempotencia, fraude o datos no validos tienen mapeos explicitos.

Flujo asincronico AMQP

1. El consumidor recibe un mensaje y valida esquema/version. 2. Mapea el payload al mismo CrearFacturaCommand. 3. Invoca CrearFacturaUseCase.ejecutar. 4. Tras confirmar el resultado durable del caso de uso, emite ack manual. Para errores temporales puede aplicar reintento acotado y dead-letter; para errores permanentes puede rechazar sin reencolar y registrar motivo. La decision de ack/retry es responsabilidad del adaptador, no del dominio. RabbitMQ distingue las confirmaciones del consumidor de los publisher confirms; los acknowledgements indican que un delivery fue procesado y puede eliminarse de la cola [2].

4. Proteccion del dominio y reglas

El caso de uso no debe duplicar ni reemplazar las decisiones de negocio: coordina su ejecucion y solicita al dominio que las aplique. Cualquier canal puede invocarlo; por tanto, las mismas invariantes rigen tanto en REST como en AMQP. La UI presenta errores y resultados, pero no decide si una factura es fraudulenta ni calcula impuestos.

Control antifraude simulado

Para un trabajo academico se propone una politica determinista y explicable. Por ejemplo, FraudePolicy puede marcar para revision o rechazar cuando: a) el total supera un umbral configurable de demostracion; b) el cliente aparece en una lista interna simulada de riesgo; c) hay lineas repetidas o cantidades/precios fuera de limites admitidos. La decision devuelve un resultado de dominio (aprobada, pendiente_revision o rechazada) con codigos de motivo; no consulta HTTP ni la UI. Los umbrales deben presentarse como datos ficticios del ejercicio, no como politica real de una entidad financiera.

Politica fiscal simulada

FiscalPolicy determina base imponible, tasa aplicable, exenciones simuladas, redondeo y total conforme a entradas de dominio (por ejemplo, categoria del producto, jurisdiccion declarada y fecha efectiva). La tasa de ejemplo debe etiquetarse como parametro didactico y no como asesoramiento ni como tasa legal vigente. Para resultados reproducibles, especificar moneda, escala decimal y regla de redondeo; evitar punto flotante binario para montos.

Secuencia del caso de uso

Paso	Accion	Proteccion / resultado
1	Verificar idOperacion existente a traves del repositorio.	Idempotencia y manejo de reintentos.
2	Construir Factura desde el comando validado.	El agregado rechaza datos estructuralmente invalidos.
3	Ejecutar FraudePolicy y FiscalPolicy.	Reglas de dominio se ejecutan siempre, sin importar canal.
4	Si procede, persistir factura y resultado.	Solo una factura aceptada se guarda como emitida; revision puede seguir un estado explicito.
5	Devolver resultado de aplicacion.	REST traduce a HTTP; AMQP decide ack/retry.

La base de datos no aplica las politicas de fraude o fiscalidad. Puede imponer restricciones tecnicas de integridad (claves unicas, no nulos) como defensa adicional, pero esas restricciones no reemplazan las decisiones del dominio.

5. Persistencia intercambiable

El puerto FacturaRepository expresa operaciones con conceptos del sistema, como guardar una Factura o buscar por idempotencia. Infraestructura aporta adaptadores PostgresFacturaRepository y OracleFacturaRepository, o InMemoryFacturaRepository para demostracion/pruebas. El patron Repository media entre objetos de dominio y mapeo de datos [3]; las implementaciones concretas permanecen fuera del nucleo.

Implementacion	Ubicacion	Seleccion	Limitaciones conocidas
PostgresFacturaRepository	Infraestructura	facturacion.persistence.provider=postgres	Driver, URL, credenciales, migraciones y SQL propios del adaptador.
OracleFacturaRepository	Infraestructura	facturacion.persistence.provider=oracle	Driver, URL, credenciales, migraciones y dialecto propios.
InMemoryFacturaRepository	Infraestructura de prueba	facturacion.persistence.provider=memory	Volatil; no demuestra durabilidad ni concurrencia de una BD real.

Ejemplo de configuracion

```
facturacion.persistence.provider=memory
# Alternativas para despliegue:
# facturacion.persistence.provider=postgres
# facturacion.persistence.provider=oracle
```

El ensamblaje en el punto de composicion lee la configuracion, construye el adaptador correspondiente y lo inyecta en CrearFacturaService. El selector pertenece a infraestructura. No se debe usar una condicion por proveedor dentro del dominio ni ramas PostgreSQL/Oracle en el caso de uso. Cambiar proveedor puede requerir ajustes de esquema, migraciones, tipos SQL y observabilidad propios; el requisito es que esas variaciones no exijan modificar las capas internas.

Prueba de sustituibilidad

Una comprobacion fuerte consiste en ejecutar el mismo conjunto de pruebas de contrato contra los tres adaptadores. Se valida que guardar y buscar conserven semantica de idempotencia, identificadores y estados acordados. La implementacion en memoria prueba la logica sin servicio de BD; no prueba rendimiento, aislamiento, transacciones ni dialecto real de PostgreSQL u Oracle. El enunciado permite simulacion y centra la evaluacion en el limite arquitectonico y la sustituibilidad.

Transacciones y consistencia

El guardado de factura debe ser atomico respecto a sus lineas y estado. Para AMQP existe una posible ventana entre guardar y confirmar el mensaje: si la aplicacion guarda y falla antes del ack, el broker puede entregar otra vez. La clave idOperacion y una restriccion unica permiten que el caso de uso sea idempotente. Si en el futuro el caso de uso publica eventos adicionales, considerar un patron Outbox y documentar la garantia de entrega; no atribuir semantica exactly-once al simple uso del broker.

6. Errores, seguridad y operacion

Las fronteras deben tratar fallos tecnicos sin contaminar las reglas del dominio. Un error de validacion de negocio puede ser una excepcion tipada o un resultado explicito; el adaptador decide su representacion HTTP o estrategia de entrega. Los detalles de stack trace y datos sensibles no se exponen en respuestas publicas ni mensajes de error.

Categoria	Ejemplo	REST	AMQP
Entrada invalida	JSON mal formado o comando sin lineas.	400 / error de validacion.	Rechazo permanente; dead-letter segun politica.
Regla de negocio	Fraude rechazado o fiscalidad invalida.	409 o 422 segun contrato API.	Registrar resultado terminal; ack o cola de rechazo definida.
Fallo transitorio	Timeout de persistencia.	503 / respuesta reintentable.	Nack/requeue acotado con backoff; evitar ciclo infinito.
Duplicado idempotente	Misma idOperacion ya procesada.	Devolver resultado previo o 200 segun contrato.	Reconocer como procesado y ack.

Controles minimos recomendados

- Validar formato y limites de tamano antes de mapear al comando; mantener las invariantes definitivas en dominio.
- Usar TLS y gestion segura de credenciales en entornos reales; no guardar secretos en el repositorio ni en logs.
- Registrar id de correlacion, idOperacion, resultado y latencia, sin datos personales o financieros innecesarios.
- Configurar ack manual, timeout, limite de reintentos y dead-letter para el consumidor; monitorear mensajes pendientes.
- Limitar accesos a colas y endpoints, y versionar los contratos de mensaje.

7. Estrategia de pruebas y evidencia

La defensa debe demostrar fronteras y comportamiento, no solo mostrar carpetas. Se recomienda una piramide breve: pruebas de dominio aisladas, pruebas del caso de uso con un repositorio falso, pruebas de adaptadores y pruebas de composicion/configuracion. Estas pruebas son un plan recomendado; las casillas deben marcarse solo tras ejecutarlas.

Prueba / evidencia	Que demuestra	Evidencia a presentar
Unitarias del dominio	Casos permitidos, fraude detectado, calculo fiscal y redondeo.	Resultados de ejecucion y casos de borde.
Caso de uso con fake	Mismo comando y mismas reglas; persistencia mediante puerto.	Pruebas sin HTTP, AMQP ni BD real.
Adaptadores REST/AMQP	Cada formato externo se mapea al contrato comun.	Request/response HTTP y mensaje/ack demostrables.
Contrato de repositorio	Intercambiabilidad semantica en memoria y adaptadores SQL.	Misma bateria de contrato por implementacion.
Seleccion por configuracion	El proveedor se elige al ensamblar la aplicacion.	Configuraciones distintas sin cambios internos.

8. Trazabilidad con el enunciado

Requisito	Decision de diseno	Como se verifica
Una logica para API y broker.	REST y AMQP invocan CrearFacturaUseCase con el mismo CrearFacturaCommand.	Diagrama, codigo de puertos y prueba que confirma una unica implementacion del caso de uso.
El caso de uso no conoce protocolo.	No importar HTTP/AMQP ni exponer objetos de transporte en contrato.	Inspeccion de dependencias y pruebas unitarias del caso de uso aisladas.
Fraude y fiscalidad en el nucleo.	FraudePolicy y FiscalPolicy son reglas de dominio.	Pruebas de dominio invocadas desde mismo caso de uso para cualquier canal.
PostgreSQL y Oracle intercambiables.	Puerto FacturaRepository; adaptadores seleccionados en infraestructura.	Configuracion alterna proveedor sin cambiar dominio/aplicacion; fake ejecuta reglas sin motor.
Cambio sin tocar dominio/aplicacion.	Detalles externos dependen de contratos internos.	Comparacion de cambios / historial o demostracion controlada entre adaptadores.

9. Ventajas reales y costos

Ventaja concreta	Costo o limite que debe explicarse
Evita duplicar la creacion de facturas entre REST y mensajeria.	Hay que mantener mapeadores y contratos externos por separado.
Reglas centrales mas faciles de probar sin servicios externos.	Se requiere invertir en pruebas para evitar que la implementacion se desvie del diseno.
Reduce el costo de cambiar el driver o el canal cuando los contratos permanecen estables.	No elimina diferencias operativas de consultas, transacciones, rendimiento ni esquemas.
Permite evolucion incremental desde monolito sin migrar a microservicios.	Mas interfaces y ensamblaje pueden ser sobrecarga en un ejercicio pequeno si no se justifican.

No es una garantia de que todo cambio sea gratis. Si el negocio cambia, se modifica dominio/aplicacion. Si una tecnologia fuerza una semantica nueva, puede exigir cambios de contrato. La ventaja es acotar el impacto de cambios externos previsibles, hacer explicitas las dependencias y mantener las politicas centrales comprobables.

10. Riesgos y mitigaciones

Riesgo	Mitigacion
Duplicar logica fiscal en controlador o SQL.	Probar que los adaptadores solo traducen y que el caso de uso siempre ejecuta politica de dominio.
Confirmar AMQP antes de guardar.	Ack manual posterior al resultado durable; idempotencia para reentrega.
Reintentos infinitos o mensajes venenosos.	Limitar reintentos, aplicar backoff y dead-letter, documentar categorias.
Confundir memoria con persistencia durable.	Rotularla como simulada y no afirmar disponibilidad, transacciones o rendimiento de BD real.
Diferencias SQL entre proveedores.	Migraciones y mapeos pertenecen a adaptadores; contrato de repositorio comun y pruebas por proveedor.

11. Guion sugerido para la defensa

1. Presentar el problema y el objetivo: proteger politicas y aislar cambios externos. 2. Mostrar el diagrama desde ambos adaptadores hasta el mismo contrato de entrada. 3. Explicar por que el consumidor HTTP o AMQP no conoce directamente el contrato interno y por que el adaptador traduce. 4. Recorrer una factura aceptada, una rechazada por fraude y el calculo fiscal. 5. Mostrar el puerto FacturaRepository y alternar proveedor en configuracion. 6. Ejecutar pruebas del dominio y caso de uso con repositorio en memoria. 7. Explicar con precision que simula la solucion y que requeriria una base real.

Preguntas que conviene responder

• ¿Cual es el punto de entrada? CrearFacturaUseCase, contrato de la capa de aplicacion. REST y AMQP son adaptadores equivalentes que traducen a su comando.

• ¿Quien conoce HTTP o AMQP? Solo los adaptadores externos. La aplicacion no recibe Request, Message ni Channel.

• ¿Donde se define el contrato de persistencia? En el interior, como abstraccion requerida por el flujo. Infraestructura implementa el puerto.

• ¿Como se cambia PostgreSQL por Oracle? En el ensamblaje/configuracion de infraestructura se elige otro adaptador compatible; dominio y aplicacion mantienen dependencias iguales.

• ¿Que prueba una implementacion en memoria? Permite ejecutar reglas y flujo sin motor; no prueba el comportamiento fisico, durabilidad o rendimiento de las bases reales.

12. Conclusion

La Arquitectura Limpia responde de forma directa a los tres objetivos del trabajo: REST y AMQP pueden reutilizar una sola logica mediante un contrato de entrada de aplicacion; las reglas antifraude y fiscales viven en el dominio y se ejecutan sin importar el canal; y la persistencia se intercambia detras de un contrato interno implementado por adaptadores de infraestructura. La configuracion del punto de composicion permite seleccionar PostgreSQL, Oracle o memoria, sin introducir esos proveedores en las politicas del sistema.

El valor no se demuestra solo con el diagrama. Se debe respaldar con codigo que respete la direccion de dependencias y pruebas que ejecuten el mismo caso de uso sin servicios externos. La implementacion en memoria cumple el alcance academico de simulacion, siempre que sus limites queden declarados. Con estas evidencias, el equipo puede defender tanto el mecanismo como las ventajas y costos reales de la arquitectura.

Referencias

[1] Robert C. Martin, “The Clean Architecture Dependency Rule,” InformIT, 2017. https://www.informit.com/articles/article.aspx?p=2832399

[2] RabbitMQ, “Consumer Acknowledgements and Publisher Confirms,” documentacion oficial. https://www.rabbitmq.com/docs/confirms

[3] Martin Fowler, “Repository,” Patterns of Enterprise Application Architecture. https://martinfowler.com/eaaCatalog/repository.html

[4] PostgreSQL JDBC, “Connection Pools and Data Sources,” documentacion oficial del driver. https://jdbc.postgresql.org/documentation/datasource/

[5] Oracle, “Data Sources and URLs,” JDBC Developer’s Guide. https://docs.oracle.com/en/database/oracle/oracle-database/19/jjdbc/data-sources-and-URLs.html

[6] ESPE, “Arquitectura de Software - Evolucion de un Sistema de Facturacion,” enunciado del trabajo practico proporcionado para esta actividad.